Astana IT University

School of Artificial Intelligence and Data Science

FINAL PROJECT REPORT

Course: Machine Learning (AMO 3208)

Topic: Machine Learning Verifier for YOLOv8n Pedestrian Detections in a Phone-Camera Danger-Zone Monitoring System

Student: Bolatov Bekdaulet, group SE-2405 (solo project, all parts done by me)

Supervisor: Mimenbayeva A. B.

Code and data: github.com/BekdauletBol/ml-final-project-pedestrian-verifier

Astana, 2026

ABSTRACT

Small real-time person detectors such as YOLOv8n make false detections, and in a danger-zone monitoring system every false “person” inside a zone becomes a false alarm. The purpose of this project is to develop and compare machine learning models that check every YOLOv8n “person” box and classify its crop as *pedestrian*, *other object* or *background*, and to use the best model in a working application for my diploma system public.safe. I built the dataset *PedCrops v1* from COCO val2017: 21,669 crops of 128×128 pixels in three balanced classes, split by photo into training, validation and test sets. The data was cleaned, normalized and augmented, and for the classical model features were extracted (HOG) and selected (ANOVA F-test, PCA). Three models of different types were implemented and compared: HOG + SVM, an own convolutional neural network with the three required hyperparameter configurations and an improved version, and MobileNetV2 with transfer learning. On the test set, HOG + SVM reached 78.4 % accuracy, the improved CNN 85.3 % and MobileNetV2 **92.8 %** (macro F1 0.928, pedestrian recall 94.1 %, 95 % bootstrap interval 92.0–93.7 %). HOG features raised the accuracy of the classical model from 48.4 % (raw pixels) to 74.5 %. As a filter for the 1,184 real YOLOv8n boxes on the test photos, MobileNetV2 removed 13.1 % of the false positives and lost only 3.8 % of the true pedestrians, while HOG + SVM and the CNN lost 22.4 % and 15.2 %. A manual check showed that most remaining “false positives” are people without a COCO label. The best model works in a web application and as a drop-in detector for public.safe.

Keywords: pedestrian detection, YOLOv8n, false positives, HOG, support vector machine, convolutional neural network, MobileNetV2, transfer learning, danger zone.

Table of Contents

Abstract

Introduction

1 Literature Review

- 1.1 Light YOLO models for pedestrians
- 1.2 Danger-zone and intrusion systems
- 1.3 Reduction of false positives and verification
- 1.4 Comparison of the reviewed works

2 Materials and Methods

- 2.1 Data Description
- 2.2 Data Preprocessing
- 2.3 Machine Learning Methods
- 2.4 Training, Validation and Testing Procedure
- 2.5 Evaluation Metrics
- 2.6 Practical Application Setup

3 Results and Discussion

- 3.1 Exploratory Data Analysis
- 3.2 Model 1: Comparison of Preprocessing Methods
- 3.3 Model 1: Comparison of Classifiers
- 3.4 Model 2: Own CNN — Three Hyperparameter Configurations
- 3.5 Model 3: MobileNetV2
- 3.6 Comparison of the Three Models
- 3.7 Practical Application: Filtering Real YOLOv8n Detections
- 3.8 Working Program: Web Application
- 3.9 Discussion

Conclusion

References

Appendix A. Source Code

INTRODUCTION

Relevance and current state of the problem. Pedestrian safety near roads, kerbs, platforms and loading areas is a serious problem, and automatic video monitoring can help people who watch these places. Modern systems use a convolutional object detector of the YOLO family: it finds people in every frame, and simple geometric rules decide whether a person is inside a dangerous zone (Mei et al., 2023; Tran et al., 2023; Feng et al., 2024). Cheap systems must run on ordinary hardware, so they use the smallest models, such as **YOLOv8n**. Small models are fast, but they make more mistakes: they sometimes say "person" for a toy, a statue, an animal or a dark corner. In a monitoring system every such box inside a zone becomes a **false alarm**, and after many false alarms the staff stop trusting the system. Research shows two ways to reduce false positives: make the detector bigger and more complex (Liu et al., 2024; Guo et al., 2025), or add a light second stage that checks the candidates (Khan et al., 2022; Jiang et al., 2024). The second way is interesting for small systems, because the detector does not need to change.

My diploma project **public.safe** is such a system: ordinary phones work as cameras, a laptop runs YOLOv8n, checks if a person stands inside a zone drawn by the operator and creates *zone_entered* events. In this final project I build and compare machine learning models that check every YOLOv8n "person" box before it reaches the zone rules.

Purpose of the project: to develop and compare machine learning models that classify the crops of YOLOv8n "person" detections into *pedestrian*, *other object* and *background*, and to use the best model as a filter of false detections in a working application.

Object of the study: image crops of people, objects and empty places that a person detector produces in photos of public places. **Subject of the study:** machine learning methods for the classification of these crops: classical machine learning with hand-made features, convolutional neural networks trained from scratch, and transfer learning.

Objectives:

1. Build a labelled dataset of crops from a reliable public source (COCO), describe it and prepare it: cleaning, normalization, augmentation and feature selection.
2. Implement three machine learning models of different types: **HOG + SVM** (classical), **an own CNN** trained from scratch, and **MobileNetV2** with transfer learning.
3. Compare at least three preprocessing methods and the three required hyperparameter configurations of the CNN.
4. Evaluate all models on a separate test set with accuracy, precision, recall, F1-score, confusion matrices and confidence intervals, and choose the best model.
5. Test the models as a filter of real YOLOv8n detections, with the same detector weights that public.safe uses.
6. Build a working program: a web application and a drop-in detector class for public.safe.

Research methods: analysis of scientific literature (2021–2025); experimental method (supervised learning with a training, validation and test split); statistical evaluation (classification metrics, bootstrap confidence intervals); comparative analysis of preprocessing methods and models.

Theoretical and methodological basis: statistical learning theory and support vector machines, feature engineering with histograms of oriented gradients (Dalal & Triggs, 2005), gradient-based training of neural networks (Goodfellow et al., 2016), transfer learning with pretrained networks (Sandler et al., 2018) and the evaluation of object detectors with intersection over union.

Scientific novelty. The verifier is a separate light model that works after an unchanged YOLOv8n, so it can be added to an existing system. It has three classes instead of two, so it also shows *what kind* of false positive was removed. It is tested not only on its own test crops, but on the real boxes of YOLOv8n, and the effect of incomplete ground-truth labels on this test is measured by a manual check.

Practical significance. The trained verifier is packed as `VerifiedDetector` with the same `detect()` method as the detector of public.safe, and as a web application that can be shown at the defense.

Structure of the report. Section 1 reviews the literature, Section 2 describes the data and the methods, Section 3 presents and discusses the results, then the conclusion and the references follow. The project is done by one student (Bolatov Bekdaulet); all parts were done by me.

1 LITERATURE REVIEW

I analysed 12 publications from 2021–2025 from MDPI, IEEE, ASCE, Springer and arXiv about pedestrian detection, light YOLO models, danger-zone intrusion systems and the reduction of false positives. All of them were checked through their DOI or arXiv page.

1.1 Light YOLO models for pedestrians

Xiao and Feng (2023) improved YOLOv8n for pedestrian tracking: Ghost convolutions made the model 40 % smaller, and SoftNMS helped with crowds. The mAP@0.5 on CrowdHuman was 79.3 % against 82.0 % for the normal YOLOv8n, so the smaller model lost some accuracy and recall. **Liu et al. (2024)** went the other way and added attention (CBAM) and BiFPN feature fusion to YOLOv8n (YOLOv8-CB): +2.4 mAP on WiderPerson (54.7 % against 52.3 %), but the speed was measured only on an RTX 4090. **Zhou et al. (2022)** designed YOLOv4-TP-Tiny for edge GPUs and got AP 58.3 % at 31 FPS on a laptop GPU. All three show that small YOLO models are a good base for real-time systems, but their accuracy on hard pedestrians is limited.

1.2 Danger-zone and intrusion systems

Yan et al. (2022) built a pedestrian intrusion warning for traffic with a pruned YOLOv5s (55.5 MB → 7.7 MB, mAP@0.5 83.4 %). Interesting detail: they needed an extra class "others" for objects that look like pedestrians, which is exactly the problem of my project. **Mei et al. (2023)** detected people in static hazardous areas on a construction site with YOLOv5 and rules on the box (precision 96.1 %, recall 90.1 %). **Tran et al. (2023)** took the zones from a 4D-BIM model, so they change with the construction schedule (precision 95.3 %, recall 83.7 %). **Feng et al. (2024)** compared YOLOv8s, YOLOv8n and YOLOv5 on 103,500 construction images (mAP@0.5 84 %, but recall only 60.5 %). In all four systems the alarm is decided directly from one YOLO pass; nobody checks if the box is really a person.

1.3 Reduction of false positives and verification

Khan et al. (2022) proposed F2DNet: a focal detection network with a light "fast suppression head" that only removes false positives (MR^{-2} 8.7 % on CityPersons, 2.2 % on Caltech). **Jiang et al. (2024)** used a second recognition network with visual, pose and attention features after the proposals of a first detector and reduced the number of false positives from 13,969 to 4,392 on Caltech, about 70 %. **Guo et al. (2025)** improved the training of a two-stage detector for a metro camera (pedestrian-sensitive training) and also reduced false positives, but the model needed 358 ms per image on a Jetson Nano. **Apurv et al. (2021)** used the design that is closest to mine: YOLOv3 trained on COCO finds people, and a fine-tuned MobileNetV2 classifies every crop (e-scooter rider / not rider) with accuracy 0.91 and F1 0.91. **Hasan et al. (2022)** showed that pedestrian detectors trained on one driving dataset generalize badly to others, while diverse web images (COCO-like data) generalize better.

1.4 Comparison of the reviewed works

Authors, year	Data	Model	Main result	Limitation
Xiao & Feng, 2023	CrowdHuman, MOT17/20	YOLOv8n + Ghost convs, OC-SORT	mAP@0.5 79.3 %, −40 % parameters	lower recall
Liu et al., 2024	WiderPerson + own images	YOLOv8-CB	mAP@0.5 54.7 % (YOLOv8n 52.3 %)	speed only on a big GPU
Zhou et al., 2022	WiderPerson, CityPersons	YOLOv4-TP-Tiny	AP 58.3 % at 31 FPS	laptop GPU only
Yan et al., 2022	own road-camera images	pruned YOLOv5s	mAP@0.5 83.4 %, 7.7 MB	small private dataset
Mei et al., 2023	construction-site video	YOLOv5 + rules	precision 96.1 %, recall 90.1 %	no check of the box
Tran et al., 2023	own site videos	YOLOv8 + 4D-BIM zones	precision 95.3 %, recall 83.7 %	needs control points
Feng et al., 2024	Roboflow, 103,500 images	YOLOv8s	mAP@0.5 84 %, recall 60.5 %	low recall
Khan et al., 2022	CityPersons, Caltech, ECP	F2DNet	MR^{-2} 8.7 % (CityPersons)	heavy backbone, GPU
Hasan et al., 2022	5 pedestrian datasets	Cascade R-CNN	MR^{-2} 12.8 % cross-dataset	only heavy detectors
Apurv et al., 2021	own car recordings	YOLOv3 → MobileNetV2	accuracy 0.91, F1 0.91	binary task, private data
Jiang et al., 2024	Caltech, CityPersons	proposals → recognition net	FP 13,969 → 4,392	needs a pose estimator
Guo et al., 2025	Caltech, CityPersons, SY-Metro	MetroNext + PSTNet	MR 24.68 % (SY-Metro)	slow on Jetson Nano

Table 1. Comparison of the reviewed works.

Strengths of existing approaches: YOLO systems are fast and already work in real time; a second stage really removes many false positives (Khan et al., 2022; Jiang et al., 2024); light architectures such as MobileNetV2 are small enough for edge devices.

Limitations: most speed results come from desktop GPUs; danger-zone systems decide the alarm from one YOLO pass without any check; false-positive suppression is usually built inside a heavy detector, so it cannot be added to an existing YOLOv8n system; many datasets are private (Yan et al., 2022; Tran et al., 2023; Guo et al., 2025), so the results cannot be repeated.

Research gap. I did not find a work that adds a small, separate verifier after an off-the-shelf YOLOv8n in a phone-camera danger-zone system, compares classical and deep models for it, and tests it on public data. This project fills this gap: it compares HOG + SVM, an own CNN and MobileNetV2 as verifiers on public COCO data and tests them on the real boxes of YOLOv8n.

2 MATERIALS AND METHODS

2.1 Data Description

Source. The dataset *PedCrops v1* was built by me in Assignment 2 from **COCO val2017** (Lin et al., 2014), a public dataset with human-made boxes for persons and 79 other object classes. I downloaded it from the Hugging Face mirror `detection-datasets/coco` (4,952 photos, 36,781 boxes, 11,004 of them persons). The COCO annotations are under the CC BY 4.0 license, the photos come from Flickr. No synthetic data is used: the background crops are real regions of the photos, chosen at random.

Size and structure. 21,669 RGB crops of 128×128 pixels in three balanced classes (7,223 each). Every crop is one sample; the target is its class. The images are stored as JPEG files in `train.zip`, `val.zip` and `test.zip` (one folder per class), and `metadata.csv` has one row with 16 variables for every crop (Table 2). As a matrix, one crop has 128 × 128 × 3 = 49,152 values (integers 0–255).

Variable	Type	Meaning
`crop_id`, `image_id`	identifier	ID of the crop and of the COCO photo
`label`, `label_id`	categorical (target)	pedestrian / other_object / background (0 / 1 / 2)
`source`	categorical (79 values)	COCO class of the box (77 object classes, `person`), or `random_window`
`x1`, `y1`, `x2`, `y2`, `box_w`, `box_h`	numeric, continuous (px)	position and size of the box in the photo
`img_w`, `img_h`	numeric, integer (px)	size of the photo
`touches_border`	boolean	the box touches the border of the photo
`split`, `file`	categorical / text	train, val or test; path of the image in the zip

Table 2. Variables of `metadata.csv`.

Class	Train	Validation	Test	Total
pedestrian	5,163	1,140	920	7,223
other_object	5,019	1,132	1,072	7,223
background	5,078	1,052	1,093	7,223
Total crops	**15,260**	**3,324**	**3,085**	**21,669**
Photos	3,437	736	738	4,911

Table 3. Number of crops per class and split (41 of the 4,952 photos gave no crop).

Second test set – real detector output. I ran YOLOv8n (Jocher et al., 2023) with the weights of public.safe and confidence 0.30 on the 738 test photos. It produced 1,184 “person” boxes

(`data/yolo_test_detections/`, with the variables `det_id`, `image_id`, `conf`, box coordinates, `max_iou_gt` and `is_tp`). This set is used only in the practical test (Section 3.7).

2.2 Data Preprocessing

Cleaning (Assignment 2). All 4,952 photos were read without errors (0 corrupted, 0 duplicate photos); 10 gray photos were converted to RGB. Boxes that cannot be classified were removed: 3,735 person boxes smaller than 48×16 px, 10,793 object boxes with the shorter side below 32 px and 3,800 object boxes that overlap a person (so that a chair with a sitting person is not labelled "other_object"). The overlap is measured by the intersection over union:

$$\mathrm{IoU}(A,B) = \frac{|A \cap B|}{|A \cup B|} \qquad (1)$$

An object is removed if IoU > 0.1 with a person or if more than 20 % of its area is covered by a person. Background crops are random windows with width/height between 0.3 and 0.7 (tall, like a person); a window had to satisfy IoU < 0.05 with every person, at most 10 % of its area covered by a person and IoU < 0.3 with every object (33,831 random windows were rejected). 1,074 near-duplicate crops were removed with an average hash, and the classes were balanced to 7,223 crops each.

Cropping and resizing. Every box of width w and height h is enlarged by 10 % on every side, padded to a square with gray colour (114, like YOLO) and resized to 128×128 with the scale factor

$$s = \frac{128}{\max(1.2\,w,\ 1.2\,h)} \qquad (2)$$

Normalization. Every input value is standardized (z-score):

$$z = \frac{x - \mu}{\sigma} \qquad (3)$$

For my CNN, μ and σ are computed per colour channel on the training crops only (μ = 0.448 / 0.427 / 0.404, σ = 0.206 / 0.200 / 0.207). MobileNetV2 uses the ImageNet values (0.485 / 0.456 / 0.406 and 0.229 / 0.224 / 0.225), because its pretrained weights expect them. For the classical model, every feature is standardized with `StandardScaler`, fitted on the training set only, so no information from the validation or test set leaks into the model.

Augmentation (CNN training only). Random horizontal flip, rotation $\alpha \in [-15°, 15°]$, scale $k \in [0.85, 1.15]$ and shift $t_x, t_y \in [-0.1, 0.1]$ (affine transformation of the pixel coordinates), and a change of brightness b and contrast c in $[0.8, 1.2]$, where m is the mean of the image:

$$\begin{pmatrix} u' \\ v' \end{pmatrix} = \frac{1}{k}\begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix}\begin{pmatrix} u \\ v \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix} \qquad (4)$$

$$x' = b\,\big(c\,(x - m) + m\big) \qquad (5)$$

Feature extraction for the classical model. A classical classifier cannot work well with 49,152 raw values, so I compare two kinds of features. *Raw pixels:* the crop is turned to gray and resized to 32×32 (1,024 values). *HOG* (Dalal & Triggs, 2005): the crop is turned to gray and resized to 64×64, and the image gradients are computed in every pixel:

$$G_x = I(u+1, v) - I(u-1, v), \qquad G_y = I(u, v+1) - I(u, v-1) \qquad (6)$$

$$m = \sqrt{G_x^2 + G_y^2}, \qquad \theta = \arctan\frac{G_y}{G_x} \in [0°, 180°) \qquad (7)$$

The image is divided into cells of 8×8 pixels; in every cell the magnitudes m are added into a histogram of 9 orientation bins. Cells are grouped into overlapping blocks of 2×2 cells, and the vector $\boldsymbol{v}$ of every block is normalized (L2-Hys: values above 0.2 are clipped, then the vector is normalized again):

$$\boldsymbol{v} \leftarrow \frac{\boldsymbol{v}}{\sqrt{\|\boldsymbol{v}\|_2^2 + \varepsilon^2}} \qquad (8)$$

With 7×7 blocks × 4 cells × 9 bins, every crop gets **1,764 HOG features**.

Feature selection. I test two methods. *ANOVA F-test* (`SelectKBest`): for every feature j the ratio of the variance between the classes to the variance inside the classes is computed, and the k features with the highest F are kept ($K = 3$ classes, N samples, n_c samples and mean $\bar{x}_c$ in class c):

$$F_j = \frac{\sum_{c=1}^{K} n_c (\bar{x}_{cj} - \bar{x}_j)^2 / (K-1)}{\sum_{c=1}^{K} \sum_{i \in c} (x_{ij} - \bar{x}_{cj})^2 / (N-K)} \qquad (9)$$

Principal component analysis (PCA): the standardized features are projected on the first q eigenvectors W_q of their covariance matrix, where q is the smallest number that keeps 95 % of the variance:

$$\boldsymbol{z} = W_q^{\top}(\boldsymbol{x} - \boldsymbol{\mu}), \qquad \frac{\sum_{j=1}^{q} \lambda_j}{\sum_{j=1}^{d} \lambda_j} \geq 0.95 \qquad (10)$$

2.3 Machine Learning Methods

Model 1 – HOG + SVM (classical machine learning)

A support vector machine looks for the separating hyperplane with the largest margin. With soft margins it solves

$$\min_{\boldsymbol{w}, b, \boldsymbol{\xi}} \frac{1}{2}\|\boldsymbol{w}\|^2 + C \sum_{i=1}^{N} \xi_i \quad \text{s.t.} \quad y_i(\boldsymbol{w}^{\top}\boldsymbol{x}_i + b) \geq 1 - \xi_i, \;\; \xi_i \geq 0 \qquad (11)$$

where C controls the trade-off between a wide margin and training errors. With a kernel, the decision function becomes a weighted sum over the support vectors; I use the RBF (Gaussian) kernel:

$$f(\boldsymbol{x}) = \sum_{i \in SV} \alpha_i y_i K(\boldsymbol{x}_i, \boldsymbol{x}) + b, \qquad K(\boldsymbol{x}, \boldsymbol{x}') = \exp\left(-\gamma \left\| \boldsymbol{x} - \boldsymbol{x}' \right\|^2 \right) \qquad (12)$$

For three classes, the linear SVM uses one-vs-rest and the RBF SVM one-vs-one voting (scikit-learn). I also compare logistic regression (softmax over linear scores, formula (19)) and a random forest of 300 decision trees (majority vote). The pipeline is: features → standardization → (selection or PCA) → classifier.

Model 2 – own convolutional neural network (`PedCNN`)

A convolution layer slides a 3×3 kernel K over the input feature maps I (with bias b):

$$(I * K)(u, v) = \sum_{c} \sum_{p=-1}^{1} \sum_{q=-1}^{1} I_c(u + p,\, v + q)\, K_c(p, q) + b \qquad (13)$$

It is followed by an activation (ReLU or tanh) and 2×2 max-pooling; the last feature maps are averaged (global average pooling):

$$\mathrm{ReLU}(x) = \max(0, x), \qquad \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}, \qquad \mathrm{GAP}_c = \frac{1}{HW} \sum_{u,v} A_c(u, v) \qquad (14)$$

Layer	Output shape (64×64 input)	Parameters
Conv 3×3, 32 filters + activation + max-pool	32 × 32 × 32	896
Conv 3×3, 64 filters + activation + max-pool	64 × 16 × 16	18,496
Conv 3×3, 128 filters + activation + max-pool	128 × 8 × 8	73,856
Conv 3×3, 128 filters + activation + max-pool	128 × 4 × 4	147,584
Global average pooling	128	0
Dense 128 → hidden (64 or 32) + activation	64 / 32	8,256 / 4,128
Dense hidden → 3 (softmax)	3	195 / 99

Table 4. Architecture of my CNN (249,283 parameters with hidden size 64).

The improved CNN adds **batch normalization** after every convolution and **dropout** in the head. Batch normalization standardizes every channel over the mini-batch $\mathcal{B}$ and then scales and shifts it with learned γ, β; dropout sets a unit to zero with probability p during training:

$$\widehat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \qquad y = \gamma \widehat{x} + \beta \qquad (15)$$

$$\tilde{h} = \frac{r\, h}{1 - p}, \qquad r \sim \mathrm{Bernoulli}(1 - p) \qquad (16)$$

The weights of the configurations are initialized with Xavier-uniform initialization:

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}}\right) \qquad (17)$$

Model 3 – MobileNetV2 (transfer learning)

MobileNetV2 (Sandler et al., 2018) is built from *inverted residual blocks*: a 1×1 convolution expands the channels, a 3×3 depthwise convolution filters every channel separately, and a 1×1 convolution projects them back, with a shortcut connection. A depthwise separable convolution needs about

$$\frac{1}{C_{out}} + \frac{1}{k^2} \qquad (18)$$

of the computation of a normal $k \times k$ convolution, which is why the network is fast on phones. I take the network pretrained on ImageNet (1.28 M images), replace its last layer by Dropout(0.2) + Linear(1280 → 3) (2.23 M parameters in total) and train it in two stages: stage 1 – only the new head (backbone frozen), stage 2 – the whole network with a small learning rate.

Output, loss and optimization (Models 2 and 3)

The network outputs three scores z_k; softmax turns them into probabilities, and the training minimizes the cross-entropy between the one-hot labels y_{ik} and the predictions p_{ik}:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{3} e^{z_j}} \qquad (19)$$

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{3} y_{ik}\log p_{ik} \qquad (20)$$

The weights θ are updated with **Adam** (Goodfellow et al., 2016); **AdamW** (Loshchilov & Hutter, 2019) adds weight decay λ directly to the update (g_t – gradient, η – learning rate):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \qquad (21)$$

$$\theta_t = \theta_{t-1} - \eta \frac{\widehat{m_t}}{\sqrt{\widehat{v_t}} + \varepsilon} - \eta\lambda\,\theta_{t-1} \qquad (22)$$

with $\widehat{m_t} = m_t/(1-\beta_1^t)$ and $\widehat{v_t} = v_t/(1-\beta_2^t)$ ($\lambda = 0$ for Adam). The improved CNN and the fine-tuning use a cosine learning-rate schedule over T epochs:

$$\eta_t = \frac{\eta_0}{2}\left(1 + \cos\frac{\pi t}{T}\right) \qquad (23)$$

A batch of 1,024 images does not fit into the 8 GB memory of my laptop, so it is processed in parts P of 256 images, and the gradients are added with weights (**gradient accumulation**). The optimizer still makes one step per batch B, so the result is the same as with a real batch of 1,024:

$$g_B = \sum_{P \subset B} \frac{|P|}{|B|} \nabla_\theta L_P \qquad (24)$$

2.4 Training, Validation and Testing Procedure

- The data is split **by photo** into training (70 %), validation (15 %) and test (15 %) sets, so crops of one photo never appear in two sets (no data leakage).
- **Training set** – fitting of the models, the scaler, the feature selection and PCA.
- **Validation set** – choice of the hyperparameters (C of the SVM, the feature method, the classifier) and **early stopping** of the neural networks: if the validation loss does not improve for 6 epochs (4 for the fine-tuning), training stops and the weights of the best epoch are restored.
- **Test set** – used only once for the final numbers.
- Training was done on my laptop (Apple M1, 8 GB, PyTorch `mps` backend); the classical models use scikit-learn on the CPU.

Model	Main hyperparameters
Model 1: HOG + SVM	HOG 64×64, 8×8 cells, 2×2 blocks, 9 bins; PCA 95 %; RBF kernel, $\gamma =$ `scale`, $C \in \{1, 10\}$
Model 2: Configs 1–3	Adam, lr 0.001, Xavier-uniform, 1 hidden layer, tanh/relu, hidden 64/32, batch 1024/128, ≤ 30 epochs
Model 2: improved CNN	ReLU, hidden 64, BatchNorm, Dropout 0.3, augmentation, AdamW lr 0.002, weight decay 1e-4, cosine, batch 128
Model 3: MobileNetV2	128×128, batch 64, AdamW, stage 1: lr 0.001, 3 epochs; stage 2: lr 0.0001, ≤ 10 epochs, cosine

Table 5. Hyperparameters of the models.

Confidence intervals. To see if the difference between two models is real, I use the bootstrap: the 3,085 test crops are resampled with replacement $B = 1{,}000$ times, the metric $\widehat{\theta}_b$ is computed every time, and the 95 % interval is given by the percentiles:

$$\mathrm{CI}_{95\%} = \left[\widehat{\theta}_{(2.5\%)},\ \widehat{\theta}_{(97.5\%)}\right] \qquad (25)$$

2.5 Evaluation Metrics

With true positives TP, false positives FP, false negatives FN and true negatives TN of one class:

$$\mathrm{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \qquad (26)$$

$$\mathrm{Precision} = \frac{TP}{TP + FP}, \qquad \mathrm{Recall} = \frac{TP}{TP + FN} \qquad (27)$$

$$F_1 = \frac{2 \cdot \mathrm{Precision} \cdot \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}, \qquad F_1^{macro} = \frac{1}{3}\sum_{k=1}^{3} F_{1,k} \qquad (28)$$

For accuracy over three classes, $TP + TN$ is the number of correctly classified crops. The macro average gives every class the same weight. For public.safe the most important numbers are the

precision and recall of the pedestrian class: low precision means false alarms, low recall means missed people. I also show confusion matrices and the loss and accuracy curves.

2.6 Practical Application Setup

For the practical test, a YOLOv8n box is a true positive if its IoU (formula (1)) with a labelled COCO person is at least 0.5, otherwise it is a false positive. The verifier keeps a box only if it is classified as pedestrian:

$$\text{keep}(b) = \begin{cases} 1, & P(\text{pedestrian} \mid \text{crop}(b)) \geq \tau \quad (\text{Models 2, 3; } \tau = 0.5) \\ 1, & \widehat{y}(\text{crop}(b)) = \text{pedestrian} \quad (\text{Model 1}) \\ 0, & \text{otherwise} \end{cases} \tag{29}$$

The working program has two parts: `src/verified_detector.py` (class `VerifiedDetector` with the same `detect()` method as the detector of public.safe) and `app.py`, a web application made with Gradio, where the user uploads a photo or takes one with the camera, chooses one of the three models and sees the kept and rejected boxes.

3 RESULTS AND DISCUSSION

3.1 Exploratory Data Analysis

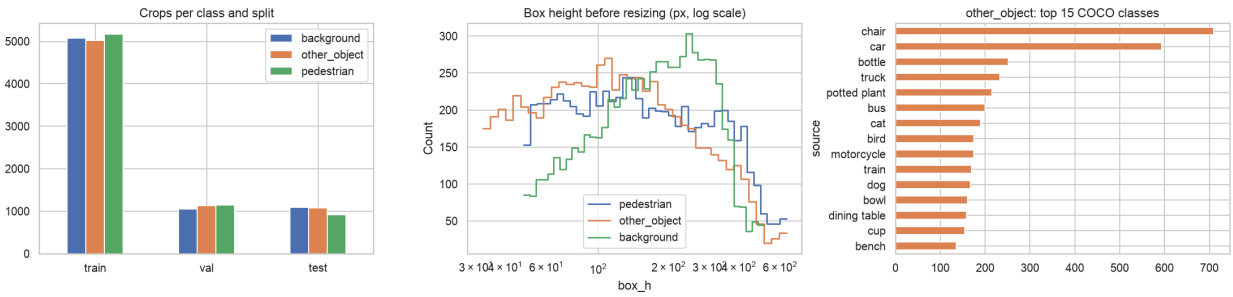


Figure 1. Left: crops per class and split. Middle: height of the boxes before resizing. Right: the 15 most frequent COCO classes in other_object.

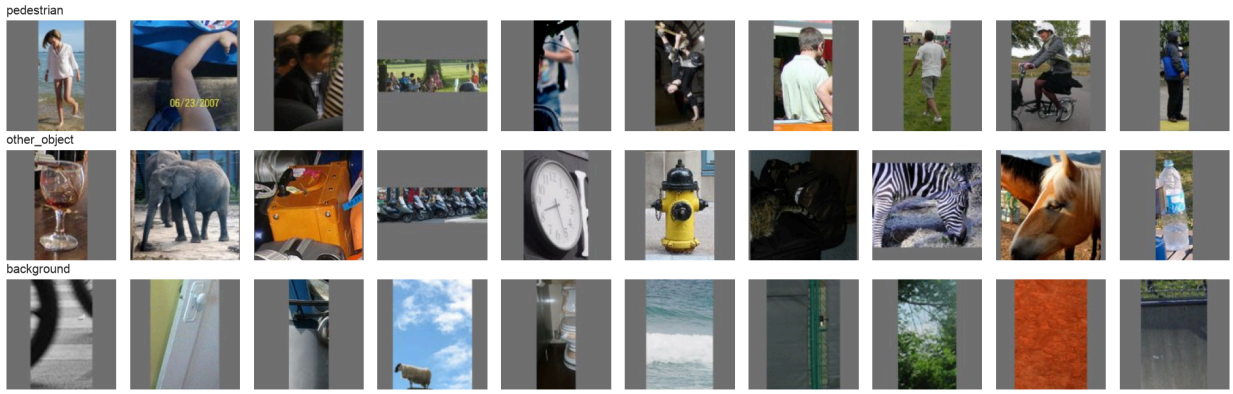


Figure 2. Random examples of the three classes.

The classes are balanced in every split (Figure 1, left). The boxes have very different sizes: the median height is 145 px for pedestrians, 111 px for other objects and 174 px for background windows, so resizing every crop to the same size is necessary. *other_object* is very diverse: 77 COCO classes, most often chairs (710), cars (594), bottles (252), trucks (233) and potted plants (215). The shape of the boxes is also different: 87.5 % of the pedestrian boxes and all background windows are taller than wide, but only 44.1 % of the object boxes. This matters later (Section 3.2).

3.2 Model 1: Comparison of Preprocessing Methods

All four feature sets were tested with the same linear SVM ($C = 0.001$, chosen on the validation set):

Preprocessing (features)	Features	Val accuracy	Val F1 (macro)	Test accuracy	Fit time (s)
Raw pixels 32×32 + linear SVM	1,024	0.487	0.486	0.484	25
HOG + linear SVM	1,764	0.739	0.739	0.745	29
HOG + ANOVA top-800 + linear SVM	800	0.721	0.720	0.738	10
HOG + PCA 95 % + linear SVM	469	0.733	0.734	0.744	8

Table 6. Preprocessing (feature) methods for the classical model, linear SVM.

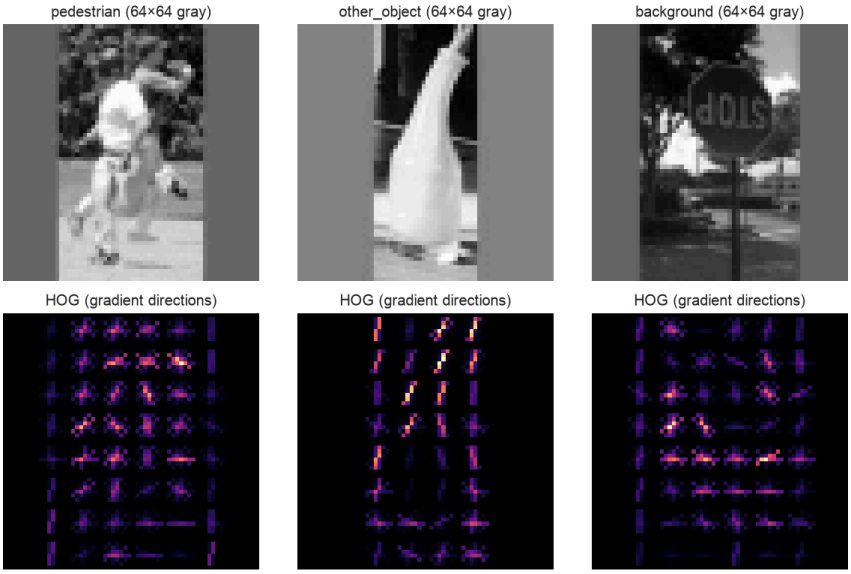


Figure 3. Gray crops and their HOG descriptors (one crop per class).

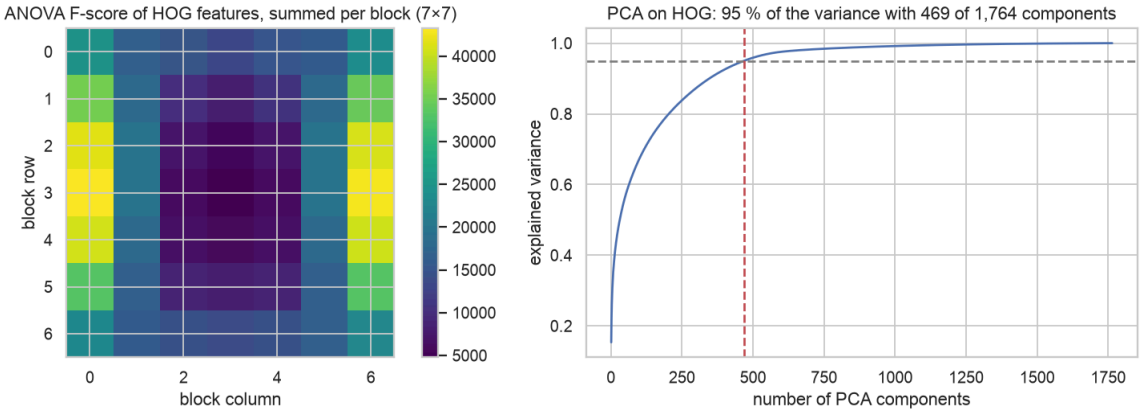


Figure 4. Left: ANOVA F-score of the HOG features, summed for every block position. Right: cumulative explained variance of PCA.

- **Raw pixels do not work** (48.4 % test accuracy, only a bit better than guessing among three classes, 33 %). A linear model on pixels cannot handle different positions, colours and

lighting.

- **HOG is the most important preprocessing step:** +26 percentage points on the test set (74.5 %). HOG describes the shape by the directions of the edges and is not sensitive to brightness (Figure 3).
- **Feature selection with ANOVA** (800 of 1,764 features) made the model 3× faster to train but lost 1.8 points on the validation set. **PCA** kept 95 % of the variance with only **469** components (Figure 4, right) and lost only 0.5 points, while the training was 4× faster. So PCA is the better way to reduce the features here.
- **What the selection shows.** The ANOVA map (Figure 4, left) shows that the most useful HOG features are at the left and right border of the crop, where the gray letterbox padding of tall boxes begins. So the shape of the box (tall or wide) is a strong cue for the classifier. This is useful, because the boxes of YOLO persons are also tall, but it also means that a part of the decision comes from the box shape, not from the content.

3.3 Model 1: Comparison of Classifiers

Classifier on HOG features	Features	Val accuracy	Val F1 (macro)	Test accuracy	Fit time (s)
HOG + PCA + logistic regression	469	0.732	0.733	0.742	7
HOG + random forest (300 trees)	1,764	0.735	0.733	0.746	39
HOG + linear SVM	1,764	0.739	0.739	0.745	29
HOG + PCA + RBF SVM (C=1)	469	0.777	0.777	0.788	42
HOG + PCA + RBF SVM (C=10) (selected)	469	0.778	0.778	0.784	96

Table 7. Classifiers on HOG features (choice by validation accuracy).

Logistic regression, random forest and the linear SVM give almost the same result (73–74 %), so a linear boundary is the limit of these features. The **RBF kernel** adds about 4 points (77.8 % validation, 78.4 % test), because it can build a non-linear boundary. $C = 1$ and $C = 10$ are almost equal on the validation set (77.7 % and 77.8 %); by the rule "best on validation" Model 1 is **HOG + PCA + RBF SVM with $C = 10$**. It keeps 10,899 support vectors, so the model file is large (44.7 MB) and the prediction is slower than for the linear models.

3.4 Model 2: Own CNN — Three Hyperparameter Configurations

Configuration	Accuracy (test)	Loss (test)	Training time	Comments
Config 1 (tanh, 64, batch 1024)	0.805	0.466	19.2 min, 30 epochs	slow start (15 updates per epoch), still improving at epoch 30 – under-trained
Config 2 (tanh, 32, batch 128)	0.805	0.482	6.6 min, 18 epochs	learns fastest (120 updates per epoch), overfits after epoch 12 (train 96 %, validation 83 %)
Config 3 (relu, 32, batch 1024)	**0.808**	**0.463**	8.6 min, 30 epochs	almost the same curves as Config 1; relu slightly better
Improved CNN (relu, BatchNorm, Dropout, augmentation, AdamW)	**0.853**	**0.370**	17.2 min, 30 epochs	no overfitting; best epoch 29

Table 8. The three required configurations and the improved CNN (test set, 3,085 crops).

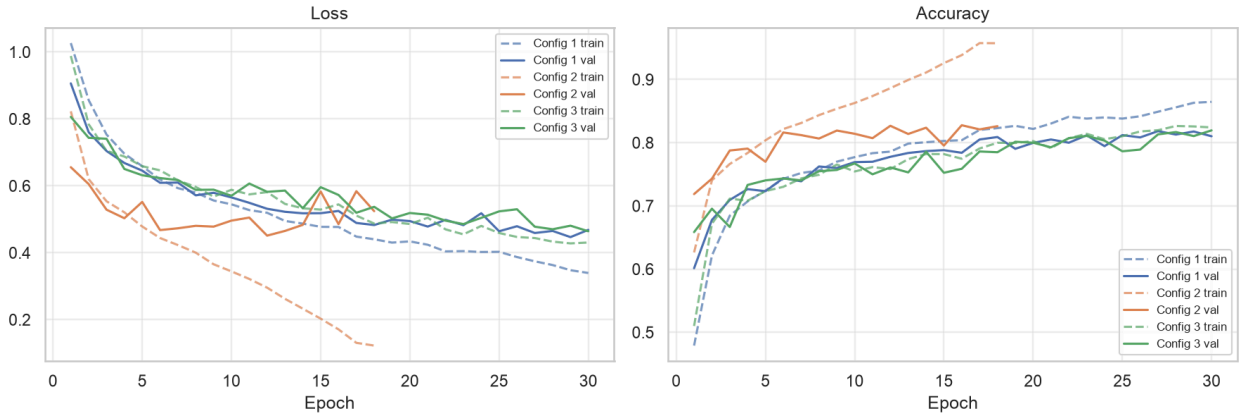


Figure 5. Loss and accuracy curves of the three configurations (dashed – training, solid – validation).

- **The three configurations reach almost the same accuracy (80.5–80.8 %).** The difference is about 10 test crops, so none of them is clearly better; without regularization the small CNN itself is the limit.
- **The batch size changed the training the most.** With batch 128 there are 120 weight updates per epoch instead of 15, so Config 2 reached 79 % validation accuracy in epoch 4 (Config 1 needed 17 epochs). But it also overfitted quickly, and early stopping ended it in epoch 18. Configs 1 and 3 had their best epoch at 29–30, so they were still learning.
- **Activation and hidden size changed little.** ReLU was 0.3 points better than tanh; the head with 64 or 32 neurons has less than 4 % of the parameters. One epoch costs about the same for all three (28–38 s when I timed them again one after another), so the different total times come from the number of epochs.
- **Regularization and augmentation** (improved CNN) raised the accuracy to **85.3 %** and removed the overfitting: training and validation accuracy stayed close (87 % and 86 %).

3.5 Model 3: MobileNetV2

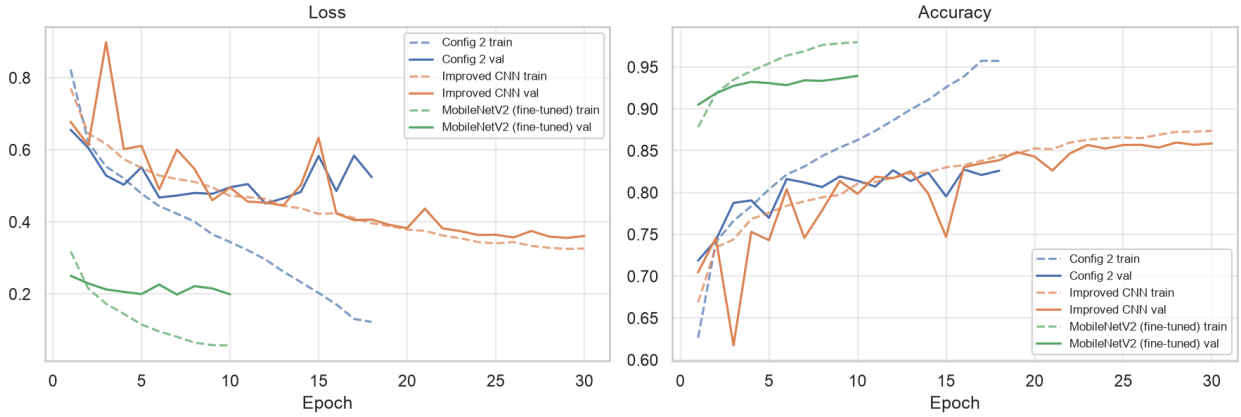


Figure 6. Curves of Config 2, the improved CNN and the fine-tuned MobileNetV2.

Training only the new head (frozen backbone) gave 83.8 % after 3 epochs. Fine-tuning the whole network started at 90.5 % validation accuracy in the first epoch, because the ImageNet features already know people and objects, and reached **92.8 %** on the test set. After epoch 7 the validation loss stopped improving while the training accuracy grew to 98 %, so early stopping restored the weights of epoch 7.

3.6 Comparison of the Three Models

Model	Accuracy (95 % CI)	Precision (macro)	Recall (macro)	F1 macro (95 % CI)	Pedestrian precision	Pedestrian recall	ms per box	File (MB)
Model 1: HOG + SVM	0.784 (0.768–0.798)	0.780	0.780	0.779 (0.763–0.794)	0.678	0.718	5.78 (CPU, 1 core)	44.7
Model 2: own CNN (improved)	0.853 (0.841–0.865)	0.851	0.851	0.850 (0.838–0.862)	0.822	0.810	1.00 (mps)	1.0
Model 3: MobileNetV2 (fine-tuned)	0.928 (0.920–0.937)	0.928	0.929	0.928 (0.920–0.937)	0.925	0.941	3.75 (mps)	9.2

Table 9. The three models on the test set. CI – 95 % bootstrap confidence interval; speed for one frame with 16 boxes.

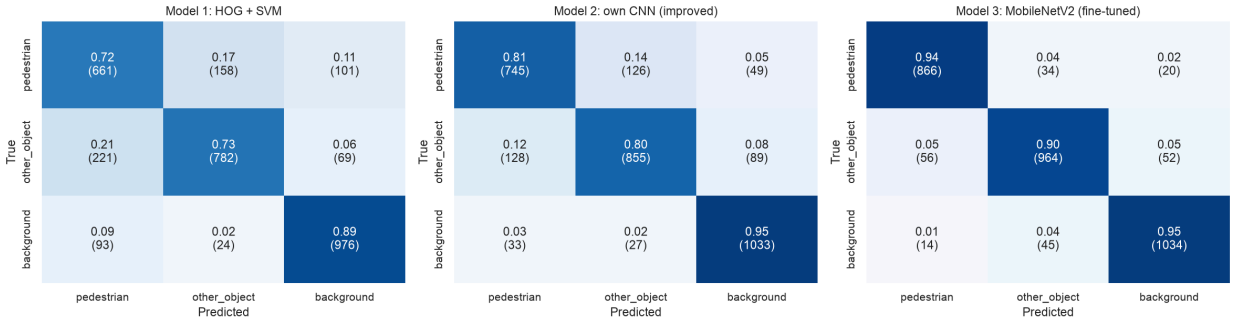


Figure 7. Normalized confusion matrices of the three models (numbers of crops in brackets).

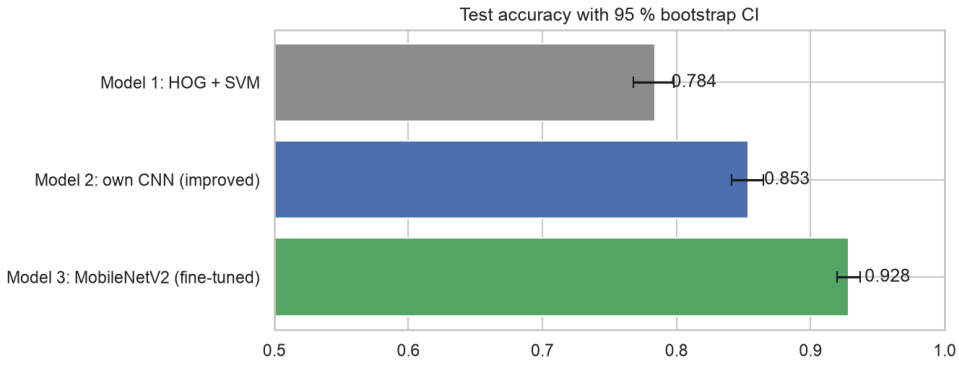


Figure 8. Test accuracy of the three models with 95 % bootstrap confidence intervals.

- **MobileNetV2 is clearly the best model:** 92.8 % accuracy and macro F1 0.928. Its confidence interval (92.0–93.7 %) does not overlap with the interval of my CNN (84.1–86.5 %). The paired bootstrap gives a difference of +7.5 percentage points (95 % CI +6.2 to +8.8). My CNN is also clearly better than HOG + SVM (+7.0 points, 95 % CI +5.6 to +8.4).
- **For public.safe** the pedestrian class matters most. HOG + SVM calls 314 of the 2,165 non-pedestrian test crops “pedestrian” (14.5 %) and misses 28.2 % of the pedestrians; the CNN – 7.4 % and 19.0 %; MobileNetV2 – only 3.2 % and 5.9 % (Figure 7).
- **The typical confusion** of all models is between *pedestrian* and *other_object*: objects near people (chairs, horses, umbrellas) often contain parts of people. The background class is the easiest (89–95 % recall).
- **Speed and size.** My CNN is the fastest (1.00 ms per box) and the smallest (1.0 MB). MobileNetV2 needs 3.75 ms per box (9.2 MB). HOG + SVM is the slowest on the CPU (5.78 ms per box) and the largest (44.7 MB), because the RBF kernel compares every crop with thousands of support vectors. All three are fast enough for a frame with several people.

3.7 Practical Application: Filtering Real YOLOv8n Detections

On the 738 test photos YOLOv8n gave 1,184 “person” boxes: 939 true positives and 245 false positives (precision 0.793, recall 0.603 against all 1,557 labelled persons). Table 10 shows the effect of each model as a filter.

Filter	True pedestrians kept	Pedestrians lost	False positives left	FP removed	Precision	Recall
YOLOv8n only	939 (100.0 %)	0.0 %	245	0.0 %	0.793	0.603
Model 1: HOG + SVM	729 (77.6 %)	22.4 %	166	32.2 %	0.815	0.468
Model 2: own CNN (improved)	796 (84.8 %)	15.2 %	195	20.4 %	0.803	0.511
Model 3: MobileNetV2 (fine-tuned)	903 (96.2 %)	3.8 %	213	13.1 %	0.809	0.580

Table 10. YOLOv8n + verifier on the 738 test photos (rule (29), threshold 0.5).

- **There is a trade-off.** HOG + SVM removes the most false positives (32.2 %), but it also throws away 22.4 % of the real pedestrians. The CNN removes 20.4 % and loses 15.2 %. **MobileNetV2 loses only 3.8 % of the pedestrians** and removes 13.1 % of the false positives. For a safety system, losing real people is worse than a false alarm, so MobileNetV2 is the only acceptable filter.
- **The precision rises only a little** (0.793 → 0.809 with MobileNetV2) and the F1 of the whole pipeline does not improve (0.685 without and 0.676 with MobileNetV2). With a higher threshold MobileNetV2 removes more false positives (22.4 % at $\tau = 0.9$) but loses more pedestrians (8.8 %).
- **Why the effect is small.** I checked the boxes by hand. 114 of the 245 false positives cover a part of a real person (IoU 0.3–0.5), so for a danger zone they are not false alarms. Among 40 random “no person” boxes that MobileNetV2 kept, **27 show a real person without a COCO label**, 5 are real mistakes (a toy figure, a dog, a bronze statue, a fire hydrant, a donkey with a cart) and 8 are too small to decide. So most “false positives” in this test are not real false alarms, and the measured gain is a pessimistic estimate.

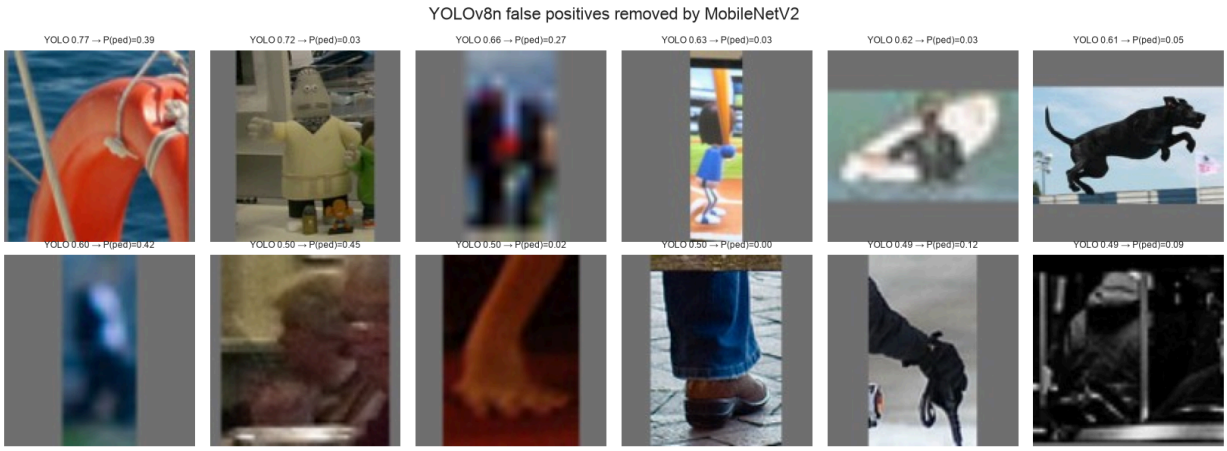


Figure 9. YOLOv8n false positives removed by MobileNetV2 (YOLO confidence, P(pedestrian)).

3.8 Working Program: Web Application

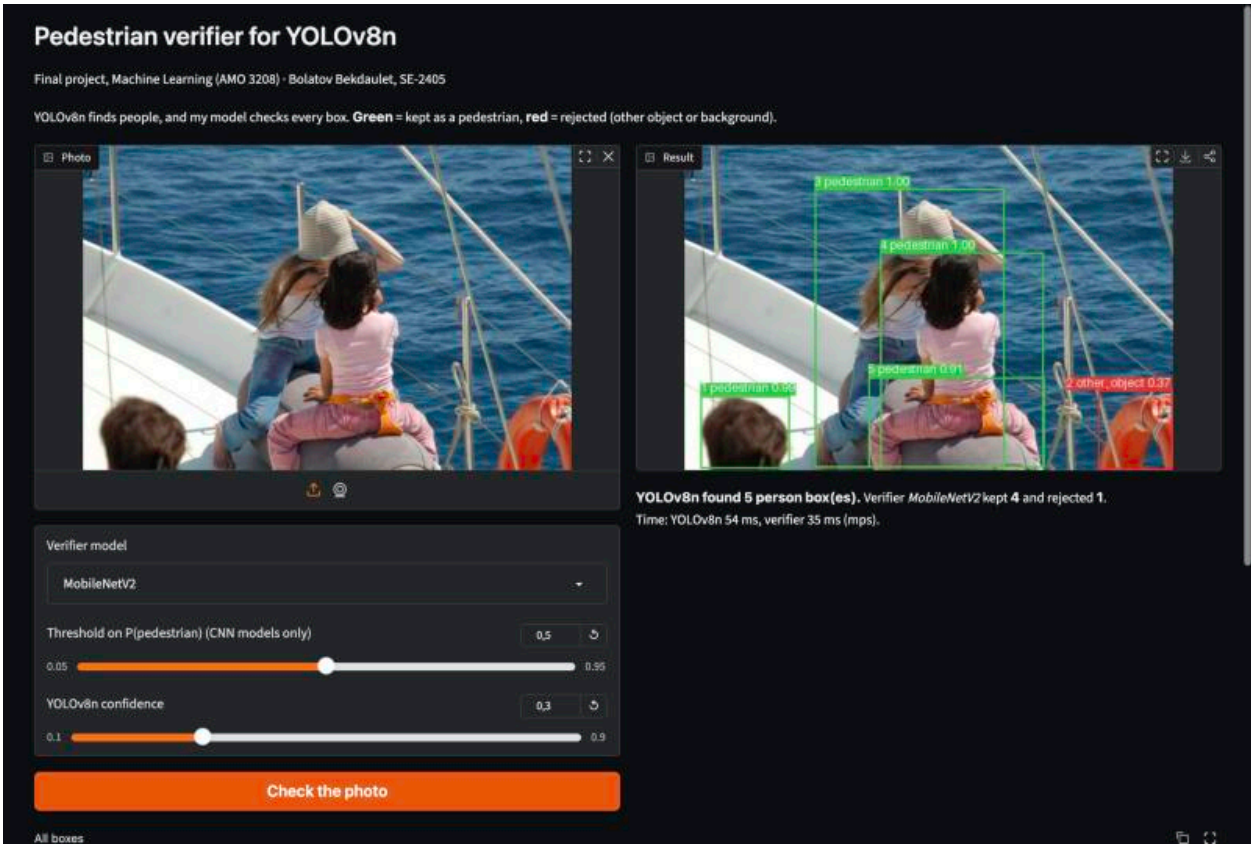


Figure 10. The web application: YOLOv8n boxes checked by MobileNetV2 (green – kept, red – rejected).

The web application (`app.py`, Gradio) runs on my laptop (`python app.py`) or in Google Colab (`--share`). The user uploads a photo or takes one with the camera, chooses the verifier (MobileNetV2, own CNN, HOG + SVM or no verifier) and the thresholds, and gets the photo with green (kept) and red (rejected) boxes, a table with the class and $P(\text{pedestrian})$ of every box, and the time of YOLOv8n and of the verifier. On the example photo of Figure 10, YOLOv8n marks a life buoy as a person; MobileNetV2 rejects it and keeps the four people. With HOG + SVM the same photo loses three real people, which shows the difference between the models in a simple way. For the diploma system, `VerifiedDetector` gives the same result through the `detect()` interface of public.safe.

3.9 Discussion

Relation to the objectives. All six objectives were reached: the dataset was built and prepared (objective 1); three models of different types were implemented (objective 2); four preprocessing methods, five classifiers and the three CNN configurations were compared (objective 3); the models were evaluated with all metrics and confidence intervals, and MobileNetV2 was chosen (objective 4); the models were tested on real YOLOv8n boxes (objective 5); and a working web application and a drop-in detector were built (objective 6).

Interpretation. The results show a clear order: hand-made features + SVM (78.4 %) < CNN trained from scratch (85.3 %) < transfer learning (92.8 %). Learned features beat hand-made

ones, and features learned on 1.28 M ImageNet images beat features learned on 15 k crops. The best preprocessing for the classical model was feature extraction (HOG, +26 points); feature selection mainly made it faster. For the CNN, regularization and augmentation were more important than the three hyperparameter sets.

Comparison with other works. Apurv et al. (2021) reached accuracy 0.91 with a MobileNetV2 crop classifier for a two-class task; my MobileNetV2 reaches 0.928 for three classes. Jiang et al. (2024) removed about 70 % of the false positives with a second network on Caltech; my filter removed 13–32 %, but in my test most "false positives" were real people without labels, so the numbers cannot be compared directly. Like Yan et al. (2022), who needed a class "others", I found that objects near people are the hardest case.

Limitations.

1. *Label noise:* not every person in COCO has a box. This affects the background class and, even more, the practical test.
2. *Domain:* COCO photos are not frames from phone cameras at a kerb or a loading area; the real false alarms of public.safe can be different (poles, bags, shadows).
3. *Shape cue:* the background windows are always tall, and the classifiers use the shape of the box (Section 3.2).
4. *One training run:* each network was trained once with one random seed; the confidence intervals show only the uncertainty of the test sample, not of the training.
5. *Recall:* every filter loses some real pedestrians; in a safety system this must be kept very low.

Further research. Record and label frames from my own phone cameras and fine-tune the verifier on them; use the false positives of YOLOv8n on the training photos as "hard negatives" in training; add boxes for the unlabelled people in COCO; combine the verifier with tracking across frames; make MobileNetV2 faster with quantization; add `VerifiedDetector` to public.safe and measure the number of false alarms per hour in a real test.

CONCLUSION

In this project I developed and compared machine learning models that check the “person” boxes of YOLOv8n, the detector of my diploma system public.safe, and built a working application with the best model.

Numbered conclusions (one for every objective):

1. **Data.** From COCO val2017 I built the dataset *PedCrops v1*: 21,669 crops of 128×128 pixels, three balanced classes of 7,223 crops, split by photo into 15,260 / 3,324 / 3,085 crops (70 / 15 / 15 %) without data leakage. Cleaning removed 18,328 unusable boxes and 1,074 duplicate crops; the inputs were normalized with the z-score.
2. **Three models.** I implemented HOG + SVM (classical, 469 PCA features, RBF kernel), an own CNN with 249 k parameters (three configurations and an improved version) and MobileNetV2 with transfer learning (2.23 M parameters).
3. **Preprocessing and configurations.** HOG features raised the accuracy of the classical model from 48.4 % (raw pixels) to 74.5 %; PCA reduced the 1,764 features to 469 with only 0.5 points less on the validation set and 4× faster training, and was better than ANOVA selection. The three CNN configurations reached 80.5–80.8 %; the batch size had the biggest effect on the training speed and overfitting, while BatchNorm, Dropout and augmentation gave +4.5 points (85.3 %).
4. **Best model.** MobileNetV2 is the best model: **92.8 % accuracy** (95 % CI 92.0–93.7 %), macro F1 0.928, pedestrian precision 92.5 % and recall 94.1 %. It is 7.5 points better than my CNN and 14.4 points better than HOG + SVM; the paired bootstrap intervals of the differences (MobileNetV2 − CNN: +6.2 to +8.8 points, CNN − HOG + SVM: +5.6 to +8.4 points) do not include zero, so the order of the models is reliable.
5. **Practical test.** On 1,184 real YOLOv8n boxes, MobileNetV2 removed 13.1 % of the false positives and kept 96.2 % of the true pedestrians (precision 0.793 → 0.809). HOG + SVM and the CNN removed more false positives (32.2 % and 20.4 %) but lost 22.4 % and 15.2 % of the real pedestrians, which is not acceptable for a safety system. A manual check showed that 27 of 40 kept “false positives” are real people without a COCO label, so the real effect is larger than the measured one.
6. **Working program.** The web application (`app.py`) shows the result of every model on any photo, and the class `VerifiedDetector` can replace the detector of public.safe without other changes; MobileNetV2 needs about 3.75 ms per box on my laptop.

Recommendations. For public.safe I recommend MobileNetV2 with the threshold 0.5 as an optional filter after YOLOv8n, but only after a test on frames from the real phone cameras. For future research: collect and label frames from the system, train with the false positives of YOLOv8n as hard negatives, correct the missing labels of COCO, and combine the verifier with tracking over several frames, so that a single uncertain frame does not decide an alarm.

REFERENCES

Apurv, K., Tian, R., & Sherony, R. (2021). *Detection of e-scooter riders in naturalistic scenes* (arXiv:2111.14060). https://arxiv.org/abs/2111.14060

Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (Vol. 1, pp. 886–893). IEEE. https://doi.org/10.1109/CVPR.2005.177

Feng, R., Miao, Y., & Zheng, J. (2024). A YOLO-based intelligent detection algorithm for risk assessment of construction sites. *Journal of Intelligent Construction, 2*(4), 9180037. https://doi.org/10.26599/JIC.2024.9180037

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org

Guo, Q., Zhang, R., & Zhang, B. (2025). A pedestrian sensitive training algorithm for false positives suppression in two-stage CNN detection methods. *Computers, Materials & Continua, 84*(1), 1307–1327. https://doi.org/10.32604/cmc.2025.063288

Hasan, I., Liao, S., Li, J., Akram, S. U., & Shao, L. (2022). *Pedestrian detection: Domain generalization, CNNs, transformers and beyond* (arXiv:2201.03176). https://arxiv.org/abs/2201.03176

Jiang, Z., Huang, S., & Li, M. (2024). A pedestrian detection network based on an attention mechanism and pose information. *Applied Sciences, 14*(18), 8214. https://doi.org/10.3390/app14188214

Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8* (Version 8.0.0) [Computer software]. https://github.com/ultralytics/ultralytics

Khan, A. H., Munir, M., van Elst, L., & Dengel, A. (2022). F2DNet: Fast focal detection network for pedestrian detection. In *2022 26th International Conference on Pattern Recognition (ICPR)* (pp. 4658–4664). IEEE. https://doi.org/10.1109/ICPR56361.2022.9956732

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In *Computer Vision – ECCV 2014* (pp. 740–755). Springer. https://doi.org/10.1007/978-3-319-10602-1_48

Liu, Q., Ye, H., Wang, S., & Xu, Z. (2024). YOLOv8-CB: Dense pedestrian detection algorithm based on in-vehicle camera. *Electronics, 13*(1), 236. https://doi.org/10.3390/electronics13010236

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *7th International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1711.05101

Mei, X., Zhou, X., Xu, F., & Zhang, Z. (2023). Human intrusion detection in static hazardous areas at construction sites: Deep learning–based method. *Journal of Construction Engineering*

and Management, 149(1), 04022142. https://doi.org/10.1061/(ASCE)CO.1943-7862.0002409

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 4510–4520). https://doi.org/10.1109/CVPR.2018.00474

Tran, S. V.-T., Lee, D., Bao, Q. L., Yoo, T., Khan, M., Jo, J., & Park, C. (2023). A human detection approach for intrusion in hazardous areas using 4D-BIM-based spatial-temporal analysis and computer vision. *Buildings, 13*(9), 2313. https://doi.org/10.3390/buildings13092313

Xiao, X., & Feng, X. (2023). Multi-object pedestrian tracking using improved YOLOv8 and OC-SORT. *Sensors, 23*(20), 8439. https://doi.org/10.3390/s23208439

Yan, X., He, Z., Huang, Y., Xu, X., Wang, J., Zhou, X., Wang, C., & Lu, Z. (2022). A lightweight pedestrian intrusion detection and warning method for intelligent traffic security. *KSII Transactions on Internet and Information Systems, 16*(12), 3904–3922. https://doi.org/10.3837/tiis.2022.12.007

Zhou, H., Wu, T., Sun, K., & Zhang, C. (2022). Towards high accuracy pedestrian detection on edge GPUs. *Sensors, 22*(16), 5980. https://doi.org/10.3390/s22165980

APPENDIX A. SOURCE CODE

The source code, the dataset and the trained models are in the project folder (and on GitHub). The notebook `Final_Project_Bolatov_Bekdaulet.ipynb` contains all steps with their outputs; its sections correspond to the models as follows.

Part	Notebook section	Files
Data loading, description, EDA	1	`src/data.py`, `data/pedcrops_v1/`, `data/build_dataset.py`
Preprocessing, HOG, feature selection	2	`src/data.py` (augmentation, normalization), `src/classical.py`
Model 1 – HOG + SVM	3	`src/classical.py`, `models/hog_svm.joblib`
Model 2 – own CNN (Configs 1–3, improved)	4	`src/models.py` (`PedCNN`), `src/train.py`, `models/pedcnn_*.pt`
Model 3 – MobileNetV2	5	`src/models.py` (`mobilenet_v2_transfer`), `models/mobilenetv2_pedcrops.pt`
Comparison of the models	6	`src/evaluate.py`
Practical application	7	`scripts/make_yolo_detections.py`, `src/verified_detector.py`, `data/yolo_test_detections/`
Web application	8	`app.py`, `src/verifiers.py`, `app_examples/`

Table A1. Which code corresponds to each model.

How to run:

```
pip install -r requirements.txt
jupyter notebook Final_Project_Bolatov_Bekdaulet.ipynb   # all results (TRAIN =
        False loads my CNN weights)
python app.py                                            # web application,
        http://127.0.0.1:7860
```